

Вертикальное федеративное обучение

Описание системы, установка и
руководство по использованию



Оглавление

- Общее описание системы, 2
- Руководство по установке, 7
- Руководство по использованию



Вертикальное федеративное обучение

(VFL+PSI) – техника машинного обучения, при которой данные из разных источников используются для обучения моделей, при этом данные остаются на стороне их владельцев.

01.

Данные остаются на стороне владельцев

Партнеры не раскрывают никакие данные для обучения



Обмен только обновлениями модели, а не данными.



Шифрование и анонимизация



Увеличение ценности данных



Снижение рисков утечек

Назначение системы

Данная система предназначена для безопасного совместного машинного обучения между двумя организациями, которые обладают непересекающимися признаками (features), но пересекающимися пользователями (одинаковыми ID).

Цель — создать модель, объединяющую информацию обеих сторон, при этом гарантируя, что никакие исходные данные одной организации не раскрываются другой.

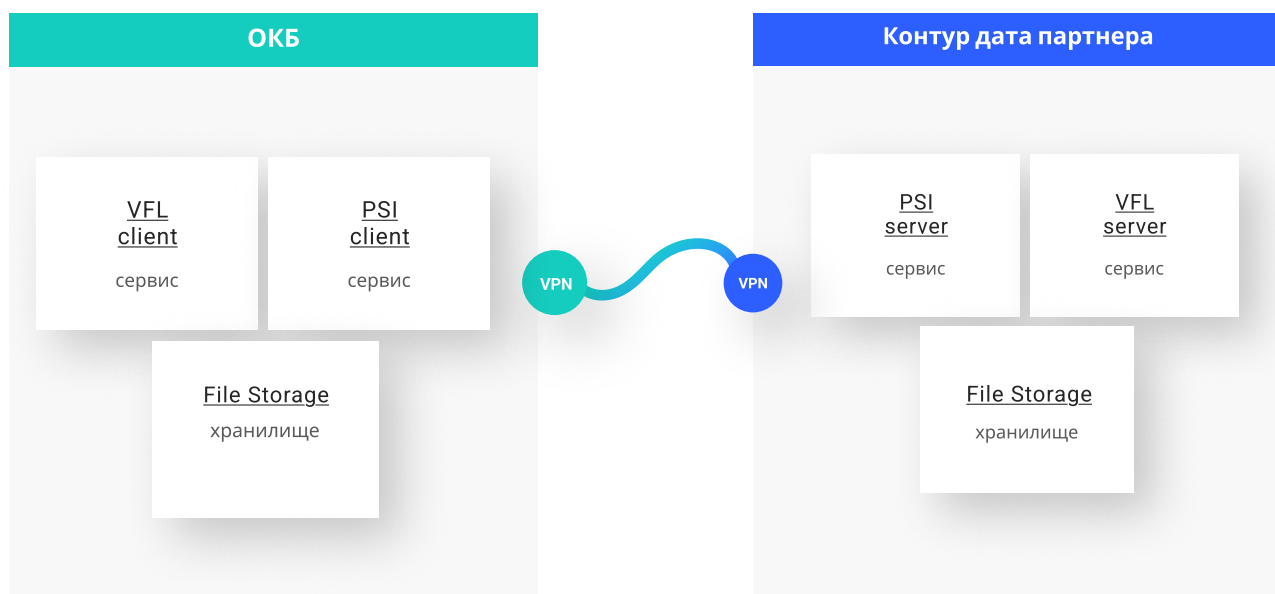
Основные технологии

Система базируется на двух ключевых технологиях:

- PSI (Private Set Intersection) — это криптографический протокол, позволяющий двум сторонам определить пересечение (общие элементы) между их наборами идентификаторов (например, user_id), не раскрывая сами наборы. Только пересечение становится известным.
- VFL (Vertical Federated Learning) — это разновидность федеративного обучения, при котором каждая сторона обладает различными признаками для одних и тех же пользователей. Например:
 - Банк хранит транзакции (X_1),
 - Дата партнер — дополнительную информацию (X_2),
 - Цель — совместно обучить модель $Y = f(X_1 \cup X_2)$, не передавая X_1 или X_2 .

Концепт архитектуры

Архитектура предполагает развертывание программного обеспечения **по обе стороны**: у дата-партнёра и у ОКБ. Участники обмениваются данными **через защищённый канал**, при этом каждая сторона использует модули PSI, вертикального федеративного обучения (VFL) для безопасной идентификации общих пользователей и совместного построения модели **без раскрытия исходных данных**.



Архитектура делится на два логических уровня:

1. Безопасное выравнивание ID (PSI Layer):

- Сначала происходит взаимное согласование — какие ID есть у обеих сторон.
- Используются криптографические схемы (например, RSA, ECC, Diffie-Hellman), чтобы ни одна сторона не получила ID другой стороны вне пересечения.
- После PSI формируется пересеченные ID, по которым строится обучение

2. Совместное обучение (VFL Layer):

- Каждая сторона подбирает значение фичей по пересеченным ID - они образуют массив для обучения
- Каждая сторона, размещает этот массив в файловое хранилище FS server на своей стороне
- Обучение происходит на основе модели градиентного бустинга, где данные каждой стороны не покидают ее контур
- Обмен градиентами (векторами минимизации loss функции) происходит в зашифрованном виде
- Централизация параметров запрещена: обучение координируется, но не консолидируется.

Участники проекта

Дата- партнер

(пассивная сторона)

- Обладает только дополнительными признаками для тех же ID (без знания целевой переменной).
- Отвечает на запросы PSI.
- Участвует в вычислениях путём передачи градиентов.

ОКБ

(активная сторона)

- Выполняет роль координатора и инициатора всех процессов.
- Хранит таргет обучения.
- Управляет запуском:
- Сессии PSI,
- Началом и завершением обучения,
- Инференсом и логированием.
- Обладает возможностью управлять процессом обучения через ручки.

Общие меры безопасности

- Все взаимодействия между сторонами проходят через закрытые каналы .
- Используется контроль контрольной суммы (SHA-512) и проверка на соответствие файлов.
- Все данные находятся в изолированных средах (Docker-контейнеры, FS-системы)
- У каждой стороны есть только своя часть данных. Даже если перехватить одну часть — она бесполезна без второй.

ID не раскрываются

Хеширование и двойное шифрование

Фичи не раскрываются

Обучение происходит локально, градиенты шифруются

Таргет не раскрывается

Хеширование и двойное шифрование

Модель не передается партнеру и ОКБ

Ни одна из сторон не видят модель полностью

Отсутствие единого датасета

Объединение данных происходит в зашифрованном виде

Передача - только зашифрованных данных

Нет утечек через файлы или сеть

PSI - пересечение hash ID

Private Set Intersection — это криптографическая процедура, позволяющая двум сторонам сравнить их наборы данных (обычно идентификаторы пользователей), чтобы определить пересекающиеся элементы, при этом не раскрывая остальную часть этих наборов.

Пример:

- Сторона А имеет ID: {1, 2, 3, 4}
- Сторона В имеет ID: {3, 4, 5, 6}
- После PSI обе стороны узнают, что пересекаются ID {3, 4} — но не узнают о других ID.

Применение PSI в системе:

- Является первым и обязательным шагом перед любым обучением.
- Является безопасной альтернативой ручной синхронизации данных по ID в Excel, Email или API.

Безопасность на этапе PSI

Что защищается?

- ID клиентов/объектов (например, номера договоров, клиентов, пользователей).
- Состав ID-сета у каждой стороны.

Как достигается безопасность:

1. Хеширование ID (обычно SHA-512) — ни одна сторона не передаёт необработанные ID. Даже если перехватить трафик — это будут только хеши.
2. Двустороннее шифрование с ключами:
 - Первая сторона шифрует свой хешированный набор ID своим ключом (например, $H(x)^{S1}$).
 - Вторая сторона шифрует свой набор своим ключом ($H(y)^{C1}$).
 - В процессе взаимодействия стороны используют дополнительные раунды шифрования, блум-фильтры, и канал, исключающий возможность утечки.
3. BloomFilter — позволяет сравнивать множества без раскрытия конкретных значений. Это структура, которая показывает "принадлежит ли элемент множеству", но не хранит сами значения.
4. Обработка сетями данных — передача ID происходит по частям, что дополнительно снижает риск анализа всего сета и атаки на пересечение.
5. Никто не видит полный ID-сет второй стороны — на выходе каждая сторона знает только пересечение, а не полные списки другой стороны.

VFL - обучение

Vertical Federated Learning — подход к обучению модели, при котором:

- Данные горизонтально пересекаются только по ID, а признаки (X) разделены.
- Каждая сторона обучает свою часть модели на своём фичевом пространстве.
- Централизованная модель строится в распределённом виде, без обмена признаками.

Преимущества:

- Каждая сторона контролирует свои данные.
- Обеспечивается соблюдение политики конфиденциальности.
- Увеличивается точность модели за счёт объединения знаний.

Безопасность на этапе VFL

Цель VFL: Совместное обучение модели, в которой:

- У одной стороны (ОКБ) есть таргет (целевая переменная).
- У второй стороны (DP) есть фичи по тем же ID.

Что защищается?

- Фичи (признаки) дата-партнёра.
- Общий обучающий датасет (в полном виде он нигде не существует).

Как достигается безопасность:

1. Каждая сторона обучает свою часть модели локально — данные не передаются.
2. Градиенты и гессианы (данные обучения) передаются в зашифрованном виде.
 - Например, шифрование используется на базе гомоморфной криптографии или других безопасных протоколов (Paillier и т.п.).
3. Модель собирается в распределённом виде — данные со сторон объединяются на уровне зашифрованных вычислений.
4. ОКБ не получает фичи партнёра — градиенты по ним зашифрованы.

Оглавление

- Общее описание системы
- Руководство по установке
- Руководство по использованию



Разграничение ролей и обязанностей

Ниже представлена детализированная процедура установки PSI + VFL с распределением шагов и ролей по каждому направлению установки:

Data Scientist ОКБ (DS_UCB)

- Формулировка исследовательских целей
- Подготовка и валидация тестовых сценариев
- Анализ результатов VFL-процессов

DevOps / Engineer ОКБ (DO_UCB)

- Развертывание инфраструктуры с UCB-стороны
- Контейнеризация PSI
- Настройка внутренней сети, CI/CD

Partner DevOps (DO_DP)

- Зеркальная установка у партнёра
- Настройка обмена сообщениями между участниками
- Управление секретами и политиками доступа

Partner Data Scientist (DS_DP)

- Подготовка данных партнёра (анонимизация, согласование форматов)
- Участие в тестировании моделей
- Интерпретация результатов

DEV_Security_UCB и DEV_Security_DP

- Аудит безопасности установки
- Настройка SSL/TLS, межсетевого экранирования
- Мониторинг и реагирование на инциденты безопасности

Architect_UCB / Architect_DP

- Согласование схемы взаимодействия (direct, VPN, reverse proxy)
- Проектируют сетевую архитектуру

Подготовка к установке

Требования к виртуальной машине (VM)

Виртуальная машина должна соответствовать следующим требованиям: Характеристики:

- ОС: Linux (Ubuntu 20.04+/Debian 10+/CentOS 7+)
- Минимальные ресурсы:
 - 4 ГБ RAM
 - 20 ГБ свободного дискового пространства

Установленное ПО:

- Docker (docker --version)
- OpenSSL (openssl version)
- Инструменты командной строки: tar, sha512sum, ssh

Сеть:

- Доступ в интернет (исходящие соединения)
- Разрешено исходящее SSH-соединение (порт 22)

01.

Подготовка

Установка
инфраструктуры

02.

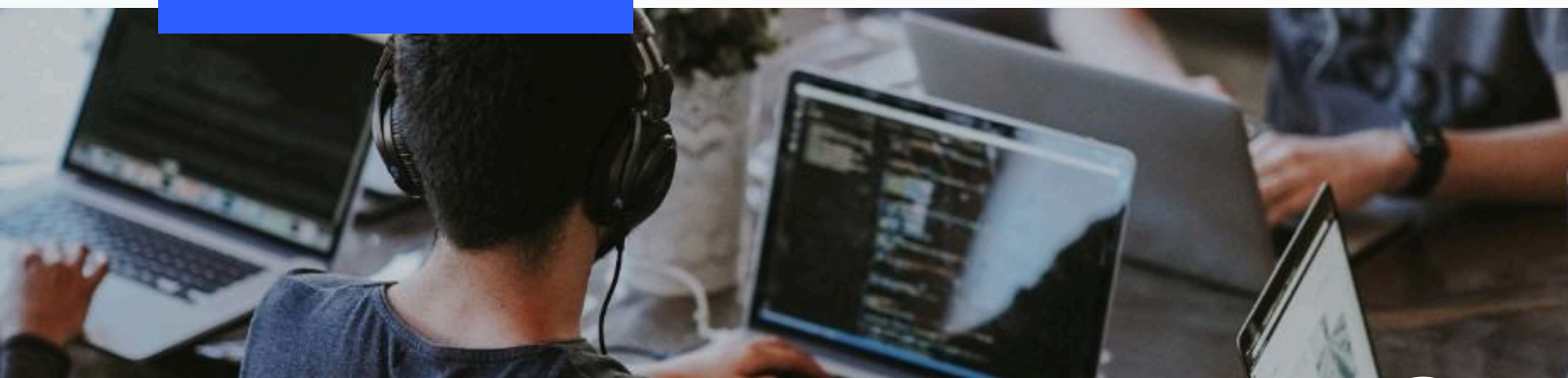
PSI

Пересечение hash-ID
для обучения без
раскрытия базы

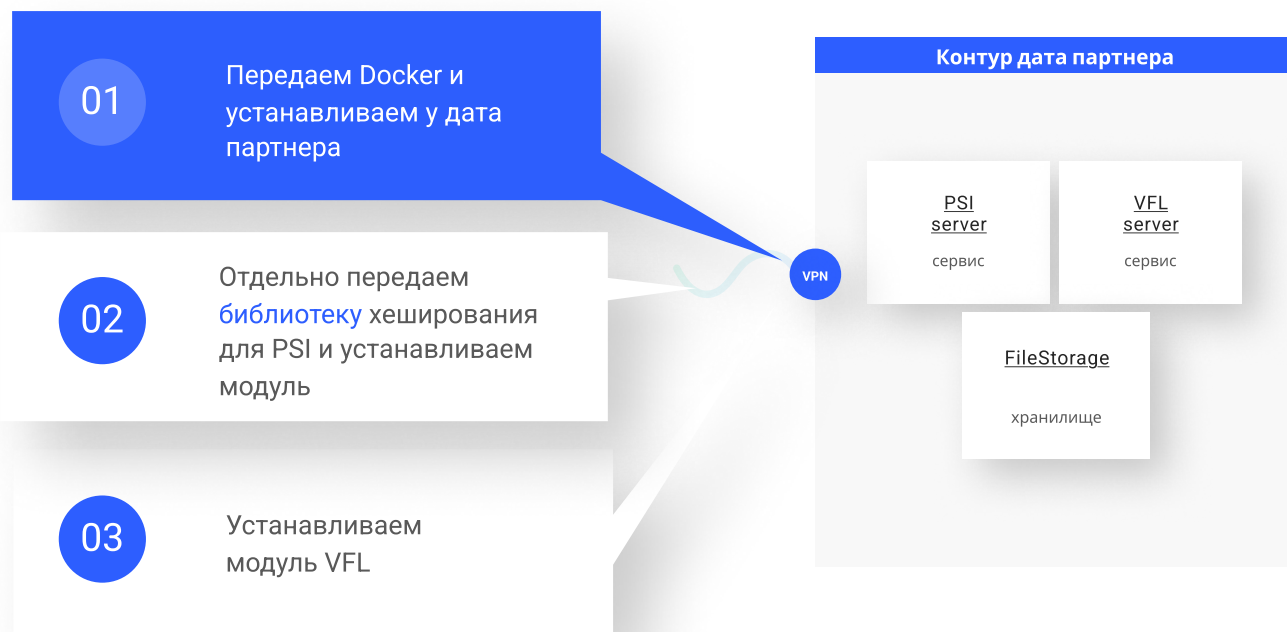
03.

VFL

Проводим вертикальное
федеративное обучение



Подготовка к установке. Docker



Этап 1: Работа с зашифрованным Docker-образом

Вам будет передано:

- `secure_image.tar.gz.enc` — зашифрованный архив Docker-образа
- `sha512sum.txt` — контрольная сумма в формате SHA-512
- Пароль — для расшифровки архива

1. Проверка контрольной суммы (SHA-512)

```
{ sha512sum secure_image.tar.gz.enc cat sha512sum.txt }
```

Убедитесь, что значения совпадают.

Если контрольная сумма не совпадает — не продолжайте и сообщите об ошибке.

2. Расшифровка Docker-архива

```
{ openssl aes-512-cbc -d -in secure_image.tar.gz.enc -out secure_image.tar.gz }
```

Введите пароль, предоставленный активной стороной.

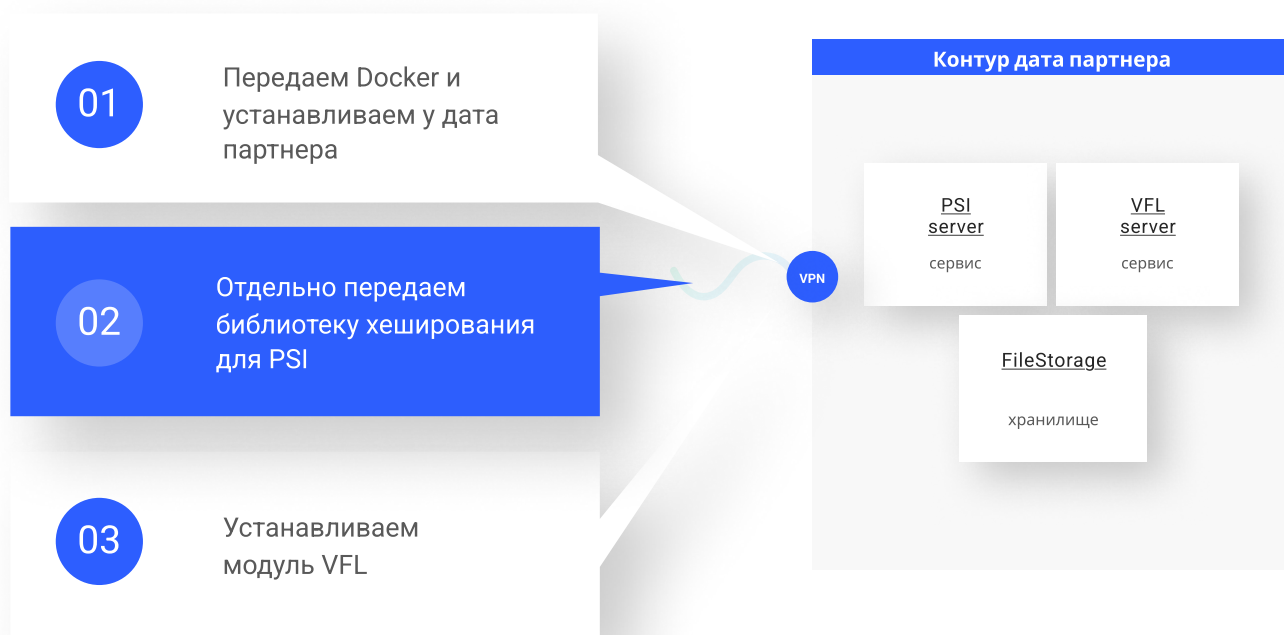
3. Распаковка архива

```
{ tar -xzf secure_image.tar.gz }
```

4. Импорт Docker-образа

```
{ docker load -i secure_image.tar }
```

Подготовка к установке.PSI



Этап 2: Подключение к активной стороне

Передаётся от активной стороны:

- IP-адрес или домен машины
- Имя пользователя
- Один из вариантов аутентификации:
 - Сертификат подключения (access_cert.pem)
 - Пароль доступа

Подключение по сертификату:

```
ssh -i access_cert.pem username@<ip_или_домен>
```

Подключение по паролю:

```
ssh username@<ip_или_домен>
```

При запросе введите пароль.

Подготовка к установке. PSI

1. Подготовка образов

Предполагается, что образы psi-server уже сохранены в файлы:

```
docker save -o psi-server.tar psi-server
```

Скопируйте эти файлы в директорию запуска (например, ~/psi-deploy/).

2. Загрузка образов

Из директории с *.tar:

```
docker load -i psi-server.tar
```

Проверить наличие образов:

Предполагается что в месте запуска у нас есть директории для клиента и сервера:

- [extdir/server/data](#)

Они будут смонтированы внутрь образа в :/mnt/ext образов.

При необходимости можно поменять их путь

4. Подготовка директорий данных

Убедитесь, что на хосте есть каталоги для монтирования:

```
mkdir -p extdir/server/data/input
```

```
mkdir -p extdir/server/data/output
```

```
mkdir -p extdir/client/data/input
```

```
mkdir -p extdir/client/data/output
```

В extdir/server/data/input/ положите файл server_dataset_hashes.txt (хеши серверных данных).

5. Запуск PSI-сервера

```
docker run -d \
```

```
  --name psi-server \
```

```
  --network psi-network \
```

```
  -v "$(pwd)/extdir/server":/mnt/ext \
```

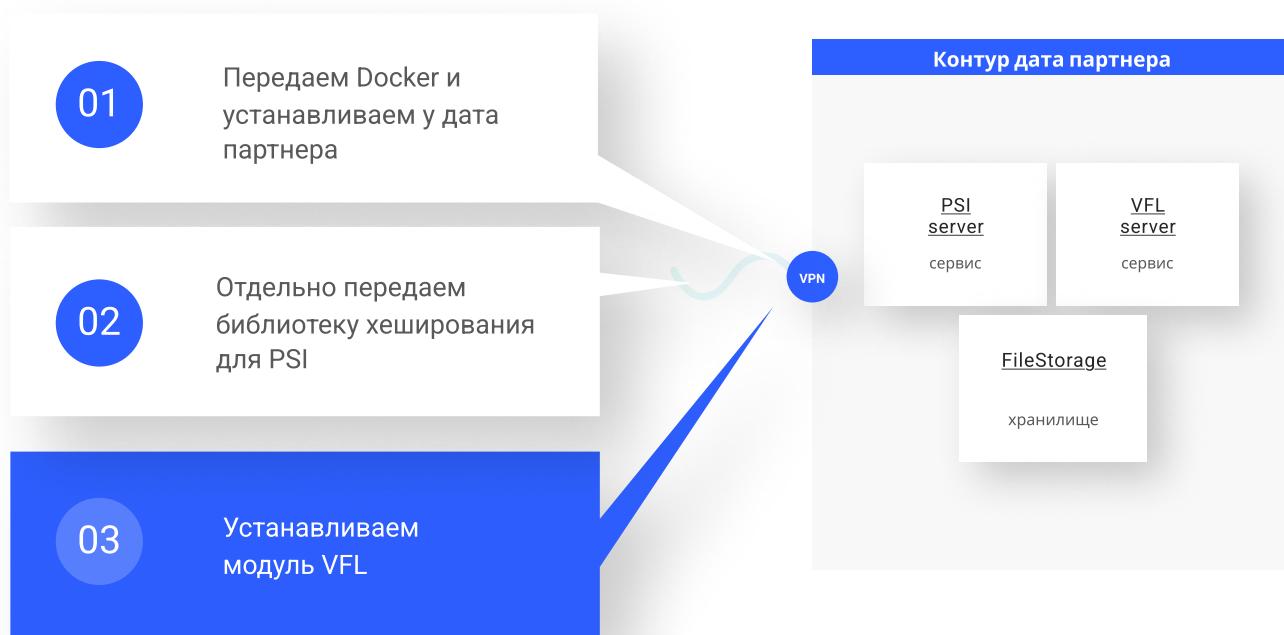
```
  psi-server \
```

```
  --input_file=/mnt/ext/data/input/server_dataset_hashes.txt \
```

```
  --output_dir=/mnt/ext/data/output \
```

```
  --listening=0.0.0.0:5051
```

Подготовка к установке. VFL



1. Загрузка образа

Если у вас есть архив образа `vfl_server-1.0.0.tgz`, выполните:

```
gunzip -c vfl_server-1.0.0.tgz | docker load
```

- `gunzip -c` распаковывает содержимое на stdout.
- Передаём поток в `docker load` для загрузки образа в локальный Docker.

Проверьте, что образ появился:

```
docker images | grep vfl_server
```

2. Запуск контейнера

```
docker run -d \
  --restart always \
  --name vfl_server \
  -v /data/vfl_server:/root/vfl_server/ \
  --network host \
  vfl_server:1.0.0 \
  /vfl/python/server.py 127.0.0.1 5672 /data/vfl_server 1
```

Конфигурация после установки. PSI

Конфигурация PSI-параметров

1. Хэш-функции

- Используется SHA-512 для всех операций хеширования набора идентификаторов.
- На вход функции подаётся строковое представление идентификатора (например, email или UUID), кодируется в UTF-8 и хешируется.
-

2. Параметры PSI

- Размер одного ID-файла: не более 10 000 строк; файлы свыше этого разбиваются на чанки по 10 000.
 - `batch_size` — размер одного чанка ID-файла, используемого для передачи и сравнения.
 - По умолчанию: 1024
 - Допустимый диапазон: 128–10 000
 - При большом объёме данных рекомендуется увеличивать, чтобы уменьшить накладные расходы сети, но не превышать возможности памяти.
- Дополнительно можно задавать:
 - `max_retries` — число попыток повторной передачи чанка при сбое (по умолчанию 3).
 - `parallelism` — число параллельных потоков обработки чанков (по умолчанию 4).

Конфигурация после установки. VFL

Общие параметры SecureBoost (список)

1. `num_trees` (int, `sys.argv[4]`)

Количество деревьев (итераций бустинга). Больше — сильнее модель, но дольше обучение.

Примеры: 100, 200

2. `max_depth` (int, по умолчанию 3)

Максимальная глубина дерева. Ограничивает сложность модели и переобучение.

Примеры: 3, 6, 10

3. `learning_rate` (float, жёстко задан, напр. 0.1)

Скорость обучения (shrinkage). Меньше — обучение медленнее, но устойчивее.

Примеры: 0.01, 0.1, 0.2

4. `objective` (string, фиксировано, напр. 'binary:bce')

Функция потерь. Например, бинарная кросс-энтропия для классификации.

Примеры: 'binary:bce', 'regression:mse'

5. `task_type` (string, часто по умолчанию 'classification')

Тип задачи: классификация или регрессия.

Примеры: 'classification', 'regression'

6. `min_child_weight` (float, задаётся в параметрах)

Минимальный вес листа. Помогает бороться с переобучением.

Примеры: 1.0, 5.0

7. `subsample` (float от 0 до 1)

Доля случайных образцов для построения дерева (bagging).

Примеры: 0.6, 0.8, 1.0

8. `colsample_bytree` (float от 0 до 1)

Доля признаков, выбираемых при построении дерева.

Примеры: 0.6, 0.8, 1.0

9. `random_state` (int, может задаваться в коде)

Фиксация случайности для воспроизводимости результатов.

Примеры: 42, 2025

Проверка установки

Поскольку пассивная сторона (Data-партнёр) не имеет внешнего API для управления процессом, валидация развёртывания проводится с помощью «smoke-тестов» на уровне контейнеров и инфраструктуры.

5.1. Статус контейнеров

Проверьте, что необходимые сервисы запущены и не перешли в аварийный статус:

```
docker ps --filter "name=psi-server" --filter "name=vfl_server"
```

- В колонке STATUS должно быть что-то вроде Up X minutes
- Если контейнер упал, используйте `docker ps -a` и `docker logs <container>` для диагностики.

5.2. Прослушиваемые порты и сеть

Убедитесь, что сервер PSI действительно слушает порт 5051, а VFL-сервер — порт 5672 (RabbitMQ), на которых они ожидалось:

```
ss -tln | grep -E "5051|5672"
```

5.3. Проверка томов и прав

Проверьте, что папки с данными смонтированы и доступны для записи:

```
docker exec psi-server ls -ld /mnt/ext/input /mnt/ext/output docker exec vfl_server ls -ld /data/input /data/output
```

И проверьте права на них:

```
docker exec psi-server stat -c "%U:%G %a" /mnt/ext docker exec vfl_server stat -c "%U:%G %a" /data
```

– Владелец должен совпадать с тем, под кем запускается процесс (обычно root или vfluser), и иметь права на запись (например, 700 или 755).

5.4. Логи на предмет ошибок

Просмотрите последние строки логов ключевых контейнеров:

```
docker logs psi-server --tail 20 docker logs vfl_server --tail 20
```

Ищите такие маркеры:

- Listening on 0.0.0.0:5051 (PSI-сервер готов)
- Waiting for tasks on RabbitMQ или Connected to rabbitmq://127.0.0.1:5672 (VFL-сервер готов)
- Отсутствие ошибок (ERROR, Traceback)

Удаление системы

Пошаговая остановка сервисов PSI + VFL (пассивная сторона)

1. Перейдите в каталог проекта

```
cd /opt/psi_vfl/
```

2. Проверьте список запущенных контейнеров

```
docker ps
```

Вы увидите что-то вроде:

```
CONTAINER ID IMAGE NAMES c1a2b3c4d5e6 vfl_runtime vfl_server d4e5f6g7h8i9 psi-
client psi_client a7b8c9d0e1f2 psi-server psi_server
```

3. Остановите контейнеры в следующем порядке:

3.1. PSI-сервер

```
docker stop psi_server
```

3.3. VFL-сервер (если используется)

```
docker stop vfl_server
```

4. Убедитесь, что контейнеры остановлены

```
docker ps
```

Если всё успешно, список будет пуст или в нём не останется контейнеров psi_/vfl_.

5. Удаление Docker-образа

```
docker rm psi-server
```

Если образ имеет тег, указывайте его, например:

```
docker rm vfl-server
```

Оглавление

- Общее описание системы
- Руководство по установке
- Руководство по использованию



Разграничение ролей и обязанностей

Роли участников

- DS_UCB (Data Scientist, активная сторона) — владеет метками, инициирует обучение
- DS_DP (Data Scientist, пассивная сторона) — подготавливает признаки
- DO_UCB (DevOps, активная сторона) — настраивает окружение, отвечает за запуск
- DO_DP (DevOps, пассивная сторона) — загружает признаки, обеспечивает передачу
- DEV_Security — контролирует безопасность, шифрование, аудит

Этапы работы с VFL + PSI

1. Подготовка

- DS_UCB + DS_DP — очищают и согласуют данные
- DO_UCB + DO_DP — загружают данные, открывают доступ
- DEV_Security — проверяет защиту и каналы

2. PSI (пересечение ID)

- DS_UCB + DS_DP — запускают PSI, получают общие ID
- DEV_Security — контролирует приватность и отсутствие утечек

3. Обучение

- DS_UCB — запускает процесс, задаёт конфигурацию
- DS_DP — передаёт признаки
- DO_UCB + DO_DP — обеспечивают связность и стабильность
- DEV_Security — проверяет шифрование и протоколы

4. Мониторинг

- DS — отслеживают метрики и логи
- DO — следят за инфраструктурой
- DEV_Security — выявляет инциденты и аномалии

5. Завершение

- DS_UCB — оценивает модель
- DO — архивируют данные
- DEV_Security — проводит финальный аудит и даёт допуск в продуктив

Подготовка данных.PSI

1. Первый круг: подготовка идентификаторов (ID)

- Цель: определить общее подмножество пользователей между активной и пассивной сторонами без раскрытия конфиденциальной информации.
- Действия:
 - Активная сторона формирует список уникальных ID из своих данных,
 - Пассивная сторона готовит ID,
 - Выполняется протокол PSI для выявления пересечения ID — только совпавшие идентификаторы используются в обучении
- Критически важно:
 - Корректность формата ID (тип данных, длина, отсутствие дубликатов).
 - Синхронизация метода sha512 хеширования/шифрования ID между сторонами

Формулирование требований к ID для выравнивания

- DS_UCB формирует технические и бизнес-требования к идентификаторам, которые должны быть использованы для пересечения.
 - Примеры: ФИО + дата рождения; паспорт; СНИЛС; email; UUID.
- DS_UCB и DS_DP согласуют схему хеширования:
 - Какие поля включаются в хеш.
 - Алгоритм хеширования (SHA-512). - важно, чтобы алгоритм хеширования совпадал у активной и пассивной части
 - Стандарты нормализации (например, регистр, удаление пробелов).

Отправка и подготовка выборки

3.1. DS_UCB отправляет предварительные требования

- Указывает:
 - Желаемые типы ID, захешированные по ранее согласованной библиотеке sha512
 - Форматы: CSV
 - Ограничения по строкам (например, до 10 000 для одного передаваемого сета ID).

Важно!

В процессе PSI применяется дополнительное шифрование захешированных ID, поэтому риск расшифровки данных отсутствует. Это гарантирует безопасность и конфиденциальность на пассивной стороне.

Подготовка данных.VFL

Цель: сформировать финальный набор данных для обучения модели

- Действия:
- ОКБ (активная сторона):
 - Отбирает признаки и таргет только для совпавших ID.
 - Контролирует целевую переменную (например, кредитоспособность, вероятность покупки)
- Дата партнер (пассивная сторона):
 - Предоставляет дополнительные признаки для пересечённых ID (например, история просмотров, демография)
 - Не имеет доступа к таргету, что исключает возможность самостоятельного обучения модели
- Пример
- После PSI остаются ID клиентов, присутствующих в обеих базах.
- ОКБ (активная сторона) использует финансовые признаки и таргет (например, одобрение кредита).
- Дата партнер (пассивная сторона) добавляет данные о покупках и поведении на сайте.

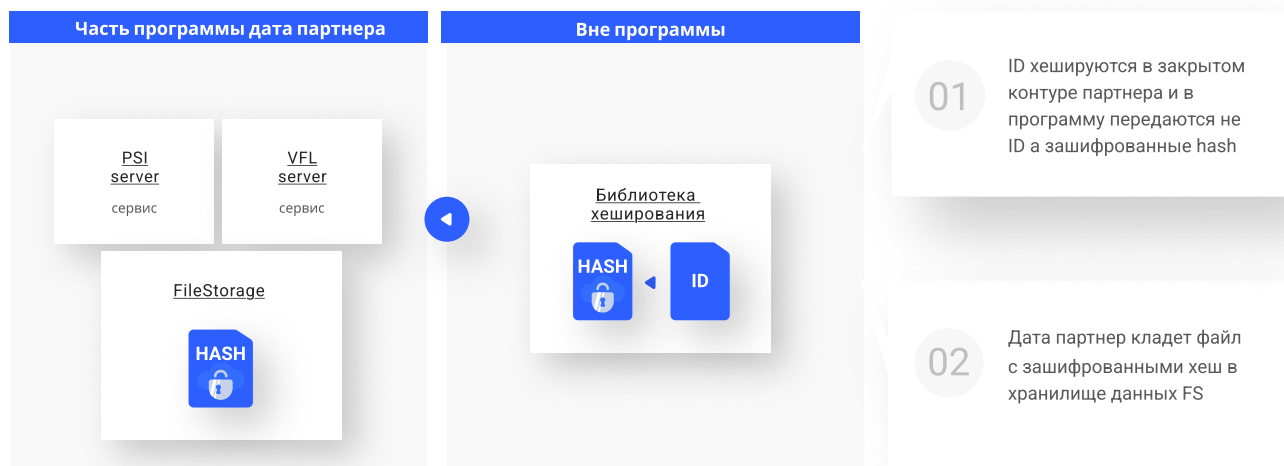
Запрос на доменные группы

- DS_UCB запрашивает у DS_DP список доступных доменных групп фичей:
- Примеры: медицинские услуги, транзакции, поведение, соц. дем., гео.
- Запрашивается уровень агрегации: помесечно, понедельно, за период.
- Запрашиваются временные интервалы доступности.

Переговоры и выбор фичей

- DS_UCB и DS_DP совместно определяют:
- Какие группы фичей будут использоваться.
- Возможный временной срез и частота (например, 6 месяцев, агрегировано по неделям).
- Какие статистики могут быть раскрыты для валидации и фильтрации (например, количество уникальных ID, среднее значение, sparsity, дисперсия).

Участие в выравнивании PSI ID



Пассивная сторона отвечает за предоставление своих ID в зашифрованном виде и участие в безопасном протоколе PSI для вычисления пересечения без раскрытия остальных данных.

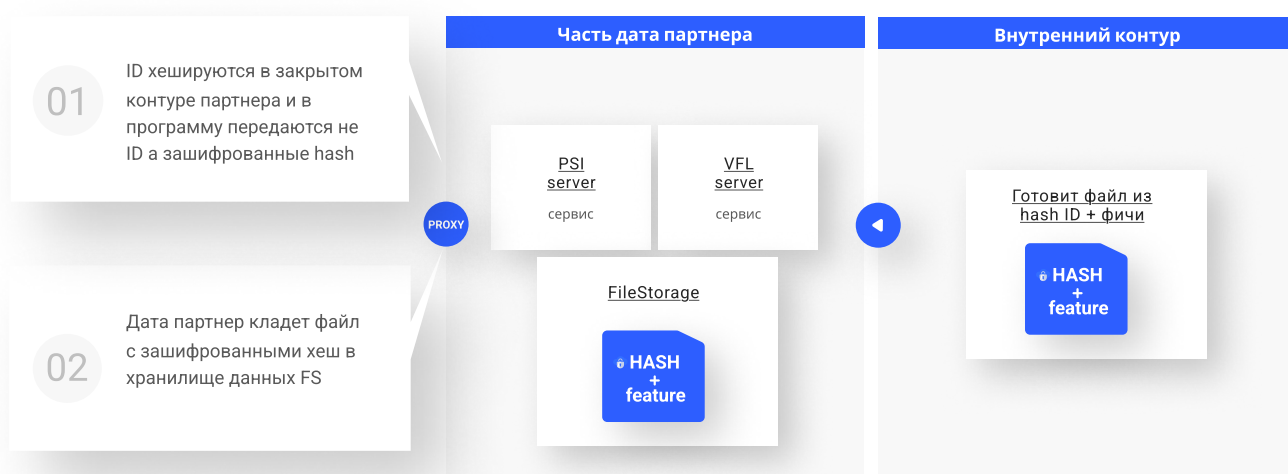
Краткое описание процесса

1. Обе стороны загружают захешированные ID в систему PSI.
2. PSI проходит в два раунда, обмениваясь зашифрованными чанками и фильтрами.
3. В результате вычисляется пересечение ID, не раскрывая их содержимого.
4. Дата партнер получает файл пересечений в output.

Инструкция для Data Partner по PSI

1. Подготовка
 - Подготовьте CSV-файл с захешированными ID по алгоритму SHA-512.
 - Убедитесь, что файл содержит только один столбец с хешами, без заголовка.
2. Размещение файла
 - Поместите файл в папку input/ на стороне PSI-сервера.
 - Пример: /mnt/ext/data/input/server_dataset_hashes.txt
 - После загрузки, сообщите активной стороне о готовности.
3. Ожидание пересечения
 - PSI-процесс запускается с активной стороны.
 - Не трогайте папку output/ до завершения пересечения.
4. Получение результата
 - Когда процесс завершён, в папке output/ появится файл с пересечёнными ID.
 - Пример: /mnt/ext/data/output/intersection_result.txt
 - Если папка пуста — значит пересечение ещё не выполнено или результат отсутствует.

Подготовка файлов VFL



Последовательность действий

1. После завершения PSI:
 - Дата-партнёр (DP) получает файл с пересечёнными ID в директории output/.
 - Эти ID при необходимости расшифровываются (если применялась схема двойного шифрования).
 - По ID формируется CSV-файл с фичами (только по пересечённым ID).
2. Размещение фичей:
 - DP размещает CSV-файл с фичами в папку input/ (например, /mnt/ext/data/input/features.csv) в контейнере PSI.
3. Сигнал ОКБ:
 - DP сообщает ОКБ (активной стороне), что файл с фичами готов и может быть использован для обучения.
4. Запуск обучения:
 - ОКБ (активная сторона) инициирует VFL-обучение.
 - Дата-партнёр (пассивная сторона) не участвует в обучении, а лишь контролирует, что система работает (например, через логи контейнера или статус-файлы).

Это гарантирует, что обучение модели происходит без раскрытия данных, с контролем целостности и конфиденциальности на всех этапах.

Инференс

Порядок действий пассивной стороны во время инференса

1. Получение запроса на инференс
Пассивная сторона получает сигнал или запрос от активной стороны (ОКБ) о необходимости запустить инференс.
2. Согласование ID для инференса
На этапе подготовки данных происходит хеширование и согласование пересечённых ID в процессе PSI, после чего определяется пул ID для инференса.
3. Подготовка данных
Пассивная сторона формирует файл с фичами, соответствующими согласованным пересечённым ID.
4. Хеширование и проверка данных
При необходимости дополнительно хеширует ID по установленному алгоритму (например, SHA-512) и проверяет целостность данных с помощью контрольных сумм.
5. Размещение данных в input-папку
Загружает подготовленный файл с фичами и хешированными ID в согласованную папку input, доступную для контейнера или системы инференса.
6. Сообщение активной стороне
Оповещает активную сторону (ОКБ), что файл с необходимыми данными размещён и готов к обработке.
7. Ожидание запуска инференса
Ожидает, пока активная сторона инициирует процесс инференса на своей стороне.
8. Контроль за процессом
Мониторит логи и состояние системы для подтверждения корректного запуска и выполнения инференса.
9. Получение результатов
После завершения инференса проверяет папку output на наличие результатов (например, предсказаний или оценок).
10. Обработка результатов
Извлекает результаты из output, проверяет их целостность и, при необходимости, передаёт результаты внутренним системам.
11. Отчётность
При необходимости сообщает активной стороне о статусе обработки или возникших ошибках.